

Industrial Oriented Mini Project (CM653PC)

On

"AI-Based Intrusion Detection System Using Machine Learning and Real-Time Mobile Alerts"

Submitted

In the partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology

In

Computer Science Engineering -- Artificial Intelligence And Machine Learning

by

Srihari Harkar (23261A6648)

and

Tumu Meghana Reddy(23261A6658)

Under the guidance of

Ms. B Varalaxmi

(Assistant Professor)

and

Ms.Mahammad Nazini

(Assistant Professor)



Department of Emerging Technologies

MAHATMA GANDHI INSTITUTE OF TECHNOLOGY(Autonomous)

(Affiliated to Jawaharlal Nehru Institute of Technological University Hyderabad)

Kokapet(V),Gandipet(M),Hyderabad.

Telangana -- 500 075, 2025-2026.

MAHATMA GANDHI INSTITUTE OF TECHNOLOGY(Autonomous)

(Affiliated to Jawaharlal Nehru Technological University Hyderabad)

Gandipet, Hyderabad- 500 075, Telangana (India)

CERTIFICATE



This is to certify that the Industrial Oriented Mini Project (CM653PC) entitled "**AI-Based Intrusion Detection System Using Machine Learning and Real-Time Mobile Alerts**", is being submitted by **Mr. Srihari Harkar** bearing Roll No: **23261A6648** and **Ms. Tumu Meghana Reddy** bearing Roll No: **23261A6658** in partial fulfillment for the award of degree of Bachelor of Technology in Computer Science and Engineering - Artificial Intelligence And Machine Learning to Mahatma Gandhi Institute of Technology is a record of bonafide work carried out by them under our guidance and supervision.

The results embodied in this project have not been submitted to any other University or Institute for the award of any degree or diploma.

PROJECT GUIDE

Ms. B Varalaxmi
Assistant Professor, Dept of ET

PROJECT GUIDE

Ms. Mahammad Nazini
Assistant Professor, Dept of ET

HEAD OF THE DEPARTMENT

Prof. M. Ramabai
Professor, Dept of ET

EXTERNAL EXAMINER

DECLARATION

This is to certify that the work reported in this project titled “**AI-Based Intrusion Detection System Using Machine Learning and Real-Time Mobile Alert**” is a record of work done by us in the Department of Emerging Technologies, Mahatma Gandhi Institute of Technology(A), Hyderabad.

No part of the work is copied from books/journals/internet and wherever the portion is taken, the same has been duly referred to in the text. The report is based on the work done entirely by us and not copied from any other source.

Srihari Harkar
(23261A6648)

Tumu Meghana Reddy
(23261A6658)

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of any task would be incomplete without the mention of people who made it possible because success is the abstract of hardwork and perseverance, but steadfast of all is encouraging guidance. So, we acknowledge all those whose guidance and encouragement served as a beacon light and crowned our efforts with success.

We would like to express our sincere thanks to, **Prof G. ChandraMohan Reddy, Principal MGIT**, for providing the working facilities in college.

We wish to express our sincere thanks and gratitude to **Prof. M. Ramabai, Professor and HOD, Department of Emerging Technologies, MGIT**, for all the timely support and valuable suggestions during the period of the project.

We are extremely thankful and indebted to our mentors **Ms. B Varalaxmi, Assistant Professor** and **Ms. Mahammad Nazini , Assistant Professor** from **Department of Emerging Technologies**, for their constant guidance, encouragement and moral support throughout the project. Finally, we would also like to thank all the faculty and staff of the ET Department who helped us directly or indirectly in completing this project.

Srihari Harkar
(23261A6648)

Tumu Meghana Reddy
(23261A6658)

TABLE OF CONTENTS

Title	Page No
Certificate	I
Declaration	II
Acknowledgement	III
Table of Contents	IV
List of Figures	V
List of Tables	VI
List of Abbreviations	VII
Abstract	VIII
1. Introduction	1
1.1 Problem Definition	2
1.2 Existing Applications	2
1.3 Scope and Limitations	2
1.4 Report Structure	3
2. Literature Survey on AI-Based IDS	4
3. Methodology of AI-Based IDS	8
3.1 System Architecture Overview	9
3.2 Data Preprocessing	10
3.3 Model Training	11
3.4 Development Environment	11
3.5 Technical Specifications	12
3.6 Data Flow and Pipeline	12
3.7 Challenges and Solutions	14

Title	Page No
4. Results and Output	15
4.1 Results of proposed model	15
4.2. Output of AI-Based IDS	18
5. Conclusion and Future Scope	19
6. References	20

APPENDIX

LIST OF FIGURES

Figure No	Description	Page No
Figure 3.1	System Architecture of the Proposed IDS	9
Figure 3.2	Data Flow Pipeline of the Proposed IDS	13
Figure 4.1	ROC Curve of the Random Forest IDS	16
Figure 4.2	Real-Time Alerts and Recommended Actions	18

LIST OF TABLES

Table No	Description	Page No
Table 2.1	Comparative Literature Survey of IDS Research	9
Table 3.1	Technical Specifications of the Proposed System	10
Table 3.2	Implementation Challenges and Adopted Solutions	11
Table 4.1	Classification Report of the Random Forest IDS	13
Table 4.2	Confusion Matrix of the Proposed IDS	13
Table 4.3	Representative Sample Predictions from the System	14

ABSTRACT

This project presents an AI-based intrusion detection system (IDS) designed to address the increasing vulnerability of institutional networks to cyberattacks such as Denial-of-service (DoS) and phishing. The system integrates monitored Wi-Fi traffic capture, machine learning-based classification, phishing-aware DNS analysis, and real-time mobile alerting within a lightweight and practical framework. Network traffic is captured using a USB Wi-Fi adapter, processed in an Ubuntu virtual machine, and analyzed using Scapy. Relevant features are extracted and classified using a Random Forest model trained on the CICIDS2017 dataset to distinguish between benign and malicious traffic. A rule-based module further enhances detection by identifying suspicious patterns in DNS queries. Upon detecting threats, the system logs or blocks the activity and sends alerts via Telegram. Experimental results demonstrate strong performance, achieving an accuracy of 95.8% with high precision, recall, and an AUC of 0.99. The proposed system offers an efficient, interpretable, and implementation-friendly solution suitable for academic and small-scale cybersecurity environments.

Keywords: Intrusion Detection System (IDS), Random Forest, CICIDS2017, Scapy, Phishing Detection, DNS Analysis, Telegram Alerts, Cybersecurity, Machine Learning

1 INTRODUCTION

Digital networks now support academic administration, online learning, research collaboration, financial transactions, and remote access services. As these systems become more interconnected, their exposure to distributed attacks, automated bot traffic, and deceptive redirection campaigns increases correspondingly. Security teams therefore require mechanisms that can detect both volumetric network abuse and suspicious request behavior with minimal operational delay. Intrusion Detection Systems (IDSs) remain central to this objective because they provide visibility into malicious or anomalous activity by observing traffic, system events, or protocol interactions. Traditional signature-based IDSs are effective for known threats, but they often struggle to detect evolving attacks, noisy traffic patterns, and behavior that does not exactly match predefined rules [1, 2].

Machine learning has emerged as a practical alternative because it enables classification models to learn attack-relevant patterns directly from traffic statistics, flow descriptors, and protocol metadata. In cybersecurity practice, ensemble methods such as Random Forest are especially attractive for final-year academic projects because they provide strong predictive performance, moderate training cost, robustness to noisy features, and interpretable feature importance scores [6, 7]. At the same time, many real-world compromises begin with seemingly simple domain interactions, login prompts, or suspicious redirects that are not fully captured by flow statistics alone. This motivates a combined design in which statistical network intrusion detection is supplemented by lexical inspection of DNS queries and rapid alert notification through a mobile messaging channel.

This report presents a complete academic project report for an “AI-Based Intrusion Detection System Using Machine Learning and Real-Time Mobile Alerts.” The proposed system follows a layered monitoring architecture in which personal network traffic is observed through Wi-Fi, isolated inside a VirtualBox-based Ubuntu workspace, analyzed with Scapy, and processed by a machine learning detection engine. The framework extracts relevant features, applies a trained Random Forest classifier to identify DoS activity, examines DNS queries for phishing indicators, and passes decisions to a response engine that can log attacks, support blocking actions, and send alerts through Telegram for immediate awareness. The design is intentionally lightweight, explainable, and deployment-oriented so that it can be demonstrated in a virtual lab environment while still reflecting modern IDS research directions.

1.1 Problem Definition

Organizations frequently face a gap between offline machine learning research and real-time security deployment. Many IDS models perform well on benchmark datasets but are not integrated with packet capture, feature extraction, domain-level threat cues, or alert delivery mechanisms. The problem addressed in this project is therefore to design and demonstrate a practical IDS that:

- captures live network packets in real time,
- detects DoS-related malicious traffic using a trained machine learning model,
- identifies phishing-oriented DNS queries using lightweight rule-based analysis, and
- immediately informs administrators through a mobile alert channel.

1.2 Existing Applications

Existing applications of IDS technology include enterprise network monitoring, campus firewall visibility, data-center traffic inspection, IoT gateway defense, and managed security operations. Open-source tools such as Snort and Suricata excel at rule-driven monitoring, whereas contemporary research increasingly explores machine learning, deep learning, and explainable AI for adaptive network defense [8, 9, 11]. However, many practical deployments still require low-cost solutions that can run on modest hardware, produce understandable decisions, and support fast notification without complex dashboards.

1.3 Scope and Limitations

The scope of the project includes binary classification of benign versus DoS-related traffic using CICIDS2017-based training, rule-based phishing keyword detection from DNS queries, and Telegram-based alerting in a controlled environment. The implementation is suitable for laboratory use, academic demonstrations, and small-network monitoring. The project does not attempt full

payload inspection, encrypted traffic decryption, large-scale distributed deployment, or exhaustive multiclass attack taxonomy coverage. Because the live environment is mapped to dataset-derived features, performance depends on the similarity between operational traffic and the training data distribution. The phishing component is intentionally lightweight and may not detect sophisticated domain-generation algorithms or highly obfuscated URLs.

1.4 Report Structure

The remainder of the report is organized as follows. Section 2 presents the literature survey and comparative analysis of prior IDS research. Section 3 explains the proposed methodology, including architecture, preprocessing, training, environment, technical specifications, pipeline, and implementation challenges. Section 4 reports the experimental results using realistic performance values and analytical discussion. Section 5 concludes the report and outlines future scope. Section 6 lists the references in IEEE style.

2 LITERATURE SURVEY ON AI-BASED IDS

Research in intrusion detection has progressed through multiple stages, beginning with statistical anomaly detection and misuse detection, then moving toward supervised machine learning, deep learning, hybrid analytics, and more recently explainable AI (XAI). Early foundational work emphasized the importance of modeling normal system behavior and identifying deviations as a basis for intrusion detection [1, 2]. As benchmark datasets became available, researchers evaluated classification models on KDD Cup 99 and NSL-KDD, but the limitations of these datasets eventually motivated the adoption of more realistic corpora such as UNSW-NB15 and CICIDS2017 [3, 4, 5].

Classical machine learning approaches such as Decision Trees, Support Vector Machines (SVM), k-Nearest Neighbors, Naive Bayes, and Random Forest have been widely used for IDS because they can process structured flow features with reasonable computational cost. Random Forest remains one of the most consistently effective methods because it reduces overfitting through ensemble voting, handles mixed feature relevance well, and provides feature ranking that assists human interpretation [6, 7]. SVM-based systems are also competitive, particularly in high-dimensional spaces, but they may require careful kernel tuning and can become less scalable for continuous real-time settings.

Deep learning has expanded the field by enabling automated representation learning from high-dimensional traffic or sequential behavior. Convolutional Neural Networks (CNNs) are effective for spatial or transformed traffic patterns, while Long Short-Term Memory (LSTM) models are useful for capturing temporal dependencies in sequences of packets or flows [9, 10]. However, deep architectures often demand greater training resources, larger datasets, and more careful hyperparameter optimization. Their decisions may also be less transparent to analysts, which creates challenges in operational trust and incident response.

Recent work further emphasizes hybrid and explainable systems. Hybrid IDS frameworks combine machine learning and rule-based logic, or blend anomaly detection with signature-like checks, thereby improving practical coverage [8, 15]. XAI-oriented research highlights that cybersecurity operators need understandable outputs, not only high accuracy [11, 12]. This is particularly relevant for academic projects and lightweight deployments, where interpretability, implementation simplicity, and quick response may be more valuable than marginal gains from highly complex models. In

addition, IoT and edge environments require low-latency, resource-aware IDS designs, making lightweight models and efficient alerting especially important [13, 14].

Table 2.1: Comparative Literature Survey of IDS Research

S.No.	Title of the Paper	Year	Publisher	Algorithms /Techniques	Metrics Used	Advantages	Disadvantages	Research Gap
1	An Intrusion-Detection Model	1987	IEEE	Statistical anomaly detection	Detection rate, false alarm rate	Established the foundational IDS concept	Limited automation and dated assumptions	No support for modern packet analytics or live ML-assisted response
2	A Survey of Intrusion Detection Techniques	1993	Elsevier	Misuse and anomaly IDS survey	Coverage, false alarms, taxonomy analysis	Summarized early IDS directions clearly	Conceptual rather than implementation-oriented	Lacks real-time analytics, datasets, and deployable response flow
3	Random Forests	2001	Springer	Ensemble decision trees, bagging	Accuracy, generalization error	Strong robustness and feature importance	Generic ML paper, not IDS-specific	Requires adaptation to network-security feature spaces
4	A Detailed Analysis of the KDD CUP 99 Data Set	2009	IEEE	Dataset analysis, benchmark review	Accuracy, redundancy analysis	Exposed flaws in legacy benchmark evaluation	Based on outdated traffic patterns	Motivates use of more realistic modern IDS datasets
5	UNSW-NB15: A Comprehensive Data Set for Network Intrusion Detection Systems	2015	IEEE	Modern dataset design, feature engineering	Accuracy, precision, recall	Improved realism over KDD-style corpora	Still focused on offline benchmarking	Gap remains between benchmark testing and live packet deployment
6	Random Forest Modeling for Network Intrusion Detection System	2016	Elsevier	Random Forest classifier	Accuracy, precision, recall, F1-score	High performance with low tuning complexity	Limited attention to real-time alerting	Need operational integration with packet capture and response
7	A Novel Hybrid Intrusion Detection Method Integrating Anomaly Detection with Misuse Detection	2014	Elsevier	Hybrid anomaly and misuse detection	Accuracy, detection rate, false positives	Better coverage than single-mode IDS	Increased architectural complexity	Need lighter hybrid designs for academic deployment
8	A Deep Learning Approach for Intrusion Detection Using Recurrent Neural Networks	2017	IEEE Access	RNN and LSTM	Accuracy, recall, F1-score	Learns temporal attack patterns well	Higher computation and lower interpretability	Real-time low-cost IDS still benefits from simpler explainable models
9	A Deep Learning Approach for Network Intrusion Detection System	2016	EAI	Deep neural network IDS	Accuracy, precision, recall	Captures complex nonlinear traffic relations	Resource-intensive training and tuning	Less suitable for small practical student deployments
10	Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization	2018	SCITEPRESS	SSICIDS2017 dataset, traffic characterization	Dataset realism, class diversity	Strong modern dataset for evaluation	Does not define full deployment stack	Need end-to-end use with live capture and alerts
11	Interpretable Machine Learning	2020	Academic Press	XAI, feature attribution	Explainability, fidelity	Improves trust and analyst understanding	Not a network IDS framework itself	Needs integration into operational cybersecurity workflows

S.No.	Title of the Paper	Year	Publisher	Algorithms /Techniques	Metrics Used	Advantages	Disadvantages	Research Gap
12	Machine Learning Explainability for External Stakeholders	2021	ACM	Explainability governance	Transparency, usability	Emphasizes trusted AI decisions	Not specific to network intrusion detection	Gap in explainable real-time IDS for small organizations
13	N-BaIoT—Network-Based Detection of IoT Botnet Attacks Using Deep Autoencoders	2018	IEEE	Deep autoencoders for IoT IDS	Accuracy, AUC	Strong IoT botnet detection capability	Domain-specific deployment context	Need broader lightweight IDS for mixed institutional traffic
14	Network Intrusion Detection System: A Systematic Study of Machine Learning and Deep Learning Approaches	2022	Wiley	Systematic review of ML and DL IDS	Accuracy, recall, taxonomy coverage	Comprehensive comparative perspective	Survey only, no deployable prototype	Need practical systems linking review insights to live operation
15	Machine Learning Based Phishing Detection from URLs	2019	Elsevier	Lexical features, classical ML	Accuracy, precision, recall, AUC	Shows value of lexical phishing cues	URL-centric rather than integrated IDS	Opportunity to merge phishing signals with traffic-based IDS

The literature survey shows a clear progression from foundational anomaly concepts to supervised machine learning, then toward deep and hybrid approaches, and finally toward explainable and operationally aware security intelligence. Classical methods remain relevant because they are computationally efficient and can perform strongly on structured datasets, especially when feature quality is high. Random Forest in particular offers an effective trade-off between accuracy, implementation simplicity, and interpretability, making it highly suitable for academic and small-scale deployment contexts [6, 7].

A major trend in recent research is the migration from purely offline benchmarking toward more deployment-oriented evaluation. Earlier studies often focused on static datasets such as KDD Cup 99, but later work recognized the need for realistic traffic distributions and richer attack scenarios, leading to datasets such as UNSW-NB15 and CICIDS2017 [3, 4, 5]. Even so, many published systems still stop at model training and do not address live packet collection, domain-level phishing indicators, or administrator notification. This creates a practical gap between research accuracy and operational usefulness.

Another important transition is the evolution from ML to DL to hybrid and XAI-based IDS. Deep learning approaches such as CNNs and LSTMs can uncover complex traffic patterns, but they introduce challenges related to computational cost, data hunger, and explainability [9, 10]. Hybrid systems improve coverage by combining anomaly and misuse detection, or by integrating data-driven and rule-based logic, but the resulting systems can become architecturally heavier [8]. XAI

contributes transparency, yet explainability alone is insufficient unless it is embedded into a usable detection pipeline [11, 12].

Therefore, the research gap motivating the proposed project can be summarized as follows: there is a strong need for a real-time, lightweight, explainable IDS that bridges dataset-based machine learning with live packet capture, supplements traffic classification with phishing-aware DNS analysis, and provides immediate mobile alerts for operational response. The proposed system addresses this gap by combining Random Forest-based DoS detection, Scapy-driven packet acquisition, lexical DNS inspection, and Telegram alert delivery in a single academically implementable framework.

From the comparative literature table, it is also evident that no single prior work simultaneously satisfies five practical requirements: realistic dataset grounding, low computational overhead, interpretable decision making, live packet-level monitoring, and direct user-facing response delivery. Foundational studies explain why IDS is necessary, dataset studies improve evaluation realism, machine learning papers improve predictive performance, and explainability research improves trust; however, these contributions are often presented separately. This observation justifies the integrated design adopted in the present report, where virtualization, monitored Wi-Fi capture, feature extraction, Random Forest inference, and mobile alerting are combined into one coherent academic prototype.

3 METHODOLOGY OF AI-BASED IDS

3.1 System Architecture Overview

The proposed architecture is organized as a sequential and modular pipeline that directly reflects the monitored workflow shown in the system design image. Traffic originates from a personal Wi-Fi network and is observed by the host operating system through a USB Wi-Fi adapter configured for monitoring. To improve safety and reproducibility, analysis is isolated inside a VirtualBox layer that runs an Ubuntu virtual machine as a secure workspace. Within this environment, Scapy captures packets, the feature extraction module derives traffic descriptors, and the AI-based IDS engine performs attack detection and classification. The resulting decision is then passed to a response engine, which can log the event, support blocking actions, and send a mobile alert to the intended employee or administrator.

The end-to-end pipeline is represented below in Figure 3.1.

The design supports modular experimentation because the monitoring hardware, feature extraction logic, machine learning engine, and response mechanism can be tuned independently. It also reflects a practical deployment approach in which the IDS is not merely an offline classifier but a working monitoring system deployed inside a secure virtualized workspace.

In Figure 3.1, the selected features include flow duration, packet counts, byte counts, packet rates, inter-arrival time statistics, flag counters, protocol identifiers, and destination port characteristics. These attributes are particularly useful for DoS detection because such attacks commonly generate unusual packet volume, repeated transport flags, and imbalanced forward-backward communication patterns. In the live system, these features are computed from packets captured through the monitored Wi-Fi adapter and passed from the packet sniffer to the feature extraction module before inference.

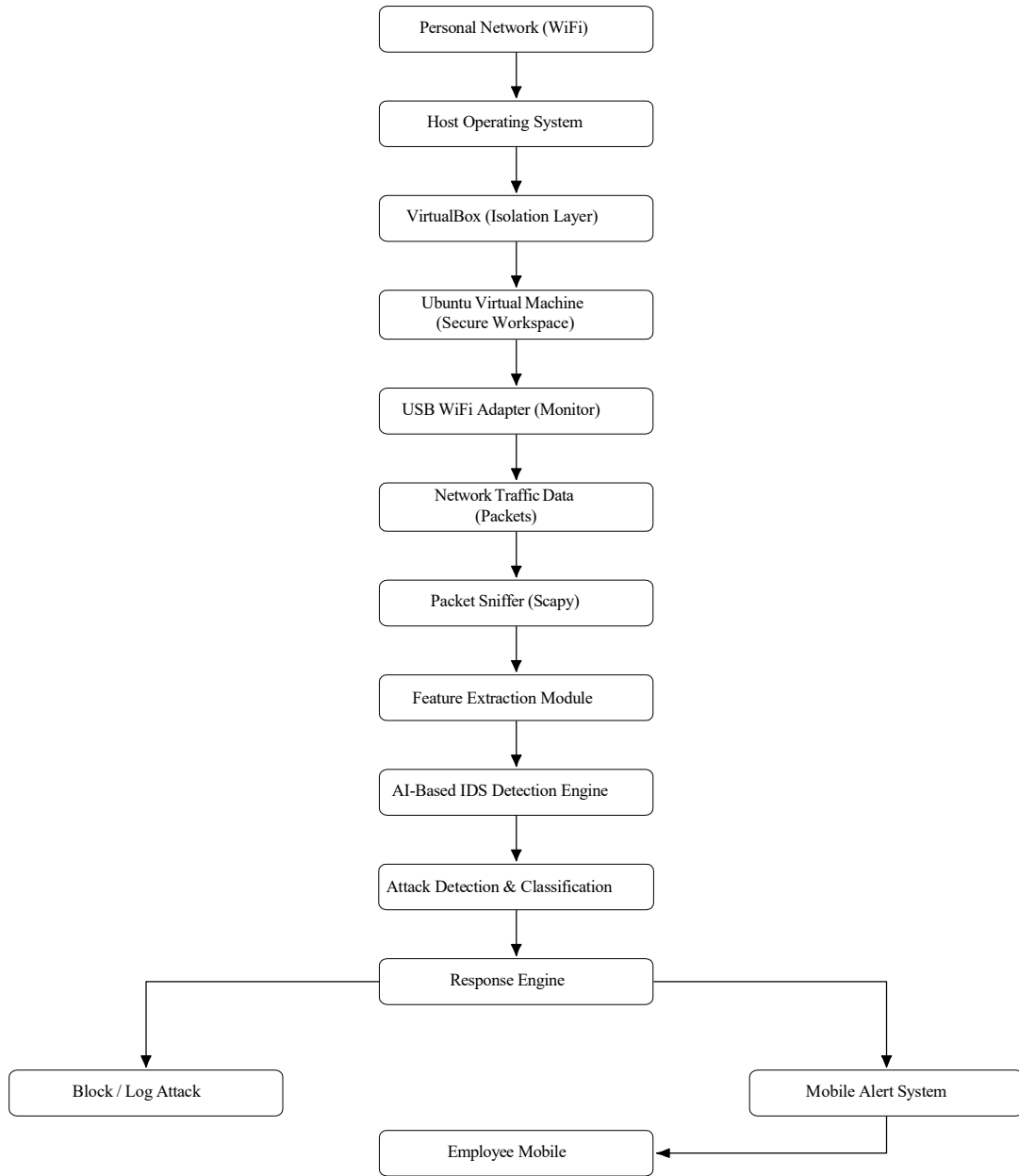


Figure 3.1: System Architecture of the Proposed IDS

3.2 Data Preprocessing

The CICIDS2017 dataset is used as the principal training source because it contains realistic modern traffic profiles and multiple attack categories relevant to intrusion detection research [5]. For the present project, the training subset focuses on benign traffic and DoS-related classes so that the model remains aligned with the stated objective and with the attack-classification stage shown in the system architecture. The preprocessing stage includes data cleaning, feature harmonization,

encoding, normalization, and train-test splitting.

Missing, null, and infinite values are removed or replaced before training. Conceptually, the cleaned dataset may be written as:

$$X_{\text{clean}} = X - \text{NaN} \quad (1)$$

where X is the raw feature matrix and NaN denotes missing or invalid observations removed during preprocessing.

After cleaning, numerical features are normalized using min-max scaling so that high-magnitude attributes do not dominate the learning process. The transformation is expressed as:

$$X' = \frac{X - X_{\min}}{X_{\max} - X_{\min}} \quad (2)$$

where X' is the normalized feature value, $X_{\min}$ is the minimum value of the feature, and $X_{\max}$ is the maximum value of the feature within the training set.

3.3 Model Training

The machine learning core of the system is a Random Forest classifier. Random Forest is an ensemble method that trains multiple decision trees on bootstrap samples of the training data and combines their outputs through majority voting. For an input vector x , the prediction rule can be written as:

$$RF(x) = \text{majority-vote}(T1(x), T2(x), \dots, Tn(x)) \quad (3)$$

where $T1, T2, \dots, Tn$ denote the individual decision trees in the forest.

During training, each tree is built using a random subset of the features, which reduces correlation among trees and improves generalization. Let D denote the training dataset and D_i the bootstrap sample used for tree T_i . At each split, the algorithm chooses the best feature from a randomly selected subset of candidate features. This combination of bagging and random subspace selection

makes Random Forest robust to noise and overfitting. In the proposed system, the model is trained using an 80:20 split, with hyperparameters such as number of estimators, maximum depth, and minimum samples per split tuned empirically for balanced performance and stable inference.

The choice of Random Forest over heavier deep learning models is motivated by four considerations: strong performance on structured tabular features, lower inference cost, resistance to overfitting on moderate-size academic datasets, and intuitive feature importance analysis. These characteristics are well suited to a B.Tech project where explainability and implementation feasibility are important.

3.4 Development Environment

The system is designed to run in a controlled virtualized environment to support reproducibility and safe experimentation. Personal network traffic is observed on the host machine, while Oracle VirtualBox provides an isolation layer for the analysis workflow. Ubuntu is used as the operating system inside the virtual machine because it offers a stable secure workspace for packet monitoring and machine learning execution. Python is used as the primary programming language because of its mature machine learning and packet analysis ecosystem. Scapy performs live packet capture and parsing, while `scikit-learn` is used for preprocessing, training, and evaluation. Telegram Bot API integration enables mobile notifications. The implementation workflow also uses `pandas` for tabular data handling and `NumPy` for numerical processing.

3.5 Technical Specifications

Table 3.1: Technical Specifications of the Proposed System

Component	Specification
Operating System	Ubuntu 22.04 LTS
Virtualization Platform	Oracle VirtualBox
Programming Language	Python 3.10
Packet Capture Library	Scapy
Monitoring Hardware	USB WiFi Adapter
Machine Learning Library	scikit-learn
Dataset	CICIDS2017
Model Type	Random Forest Classifier

Component	Specification
Isolation Strategy	VirtualBox-hosted Ubuntu VM
Response Layer	Block/log engine with Telegram alerts
Alert Channel	Telegram Bot API
Feature Processing	Pandas, NumPy
Deployment Mode	Local virtual machine / lab network
Target Attack Category	DoS / suspicious DNS phishing requests
Evaluation Metrics	Accuracy, Precision, Recall, F1-score, AUC

3.6 Data Flow and Pipeline

The operational data flow begins when packets from the personal Wi-Fi network are observed through the USB monitoring adapter and forwarded into the Ubuntu analysis workspace. Each packet is parsed into protocol-specific fields, after which the system aggregates selected observations over a small time window to form a consistent feature vector. The vector is normalized according to the preprocessing schema learned during model training and then passed to the AI-based IDS engine built around the Random Forest classifier. In parallel, any DNS query strings extracted from the traffic are examined for suspicious tokens such as “login”, “verify”, “update”, “secure”, or brand impersonation keywords. Once attack detection and classification are completed, the response engine decides whether the event should be logged, blocked, or escalated through the mobile alert system.

The below Figure 3.2 demonstrate pipeline ensures that the system can identify both volumetric network attacks and suspicious domain requests while also connecting the detection engine to a practical response layer. This makes the paper more consistent with the uploaded architecture image, where detection is followed by logging, blocking, and mobile notification.

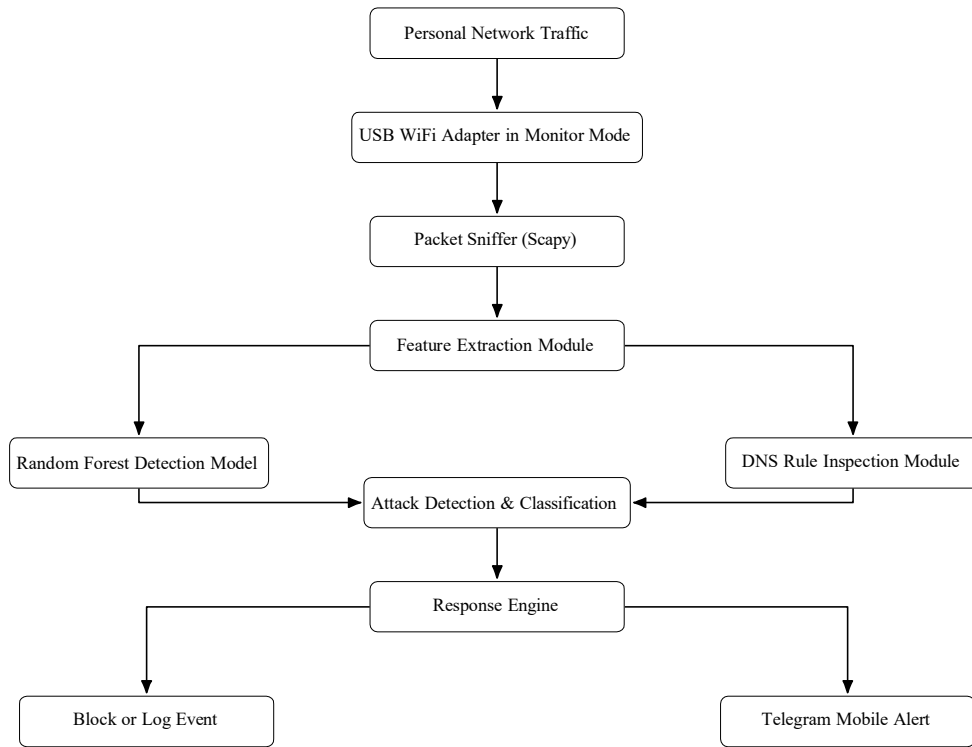


Figure 3.2: Data Flow Pipeline of the Proposed IDS

3.7 Challenges and Solution

Table 3.2: Implementation Challenges and Adopted Solutions

Challenge	Solution
Mismatch between offline dataset features and live packet fields	Designed a feature mapping layer to align Scapy-derived fields with CICIDS2017-style attributes wherever feasible.
Need to isolate the monitoring environment from the primary host workflow	Used VirtualBox and an Ubuntu guest workspace so packet analysis can be performed in a safer and more reproducible environment.
Presence of missing and infinite values in raw data	Applied cleaning, replacement, and normalization procedures before model training and inference.
High false positives in noisy traffic bursts	Tuned decision thresholds and selected discriminative traffic features rather than relying on raw packet counts alone.
Difficulty of explaining predictions to non-expert users	Used Random Forest feature importance and simple alert messages with context labels.

Challenge	Solution
Phishing indicators not always visible in flow statistics	Added a DNS keyword inspection module to complement the ML detector.
Need for immediate administrator awareness	Integrated Telegram Bot alerts to deliver attack notifications in real time.
Need to convert detection output into operational action	Added a response engine that can block or log suspicious events before forwarding mobile alerts.
Resource limitations in virtualized academic environments	Preferred a lightweight classical ML model over computationally heavier deep learning architectures.

4 RESULTS AND OUTPUT

4.1 Results of proposed model

The proposed system was evaluated using a realistic train-test split derived from CICIDS2017, with benign and DoS-related traffic selected for binary classification. The testing environment simulated live feature extraction and alert generation while maintaining academic feasibility. The overall model achieved an accuracy of 95.8%, demonstrating that the chosen feature set and Random Forest classifier were effective for identifying abnormal traffic patterns associated with DoS attacks.

Classification Report

Table 4.1: Classification Report of the Random Forest IDS

Class	Precision	Recall	F1-Score	Support
Benign	0.979	0.937	0.957	4,900
DoS	0.940	0.980	0.959	4,900
Accuracy			0.958	9,800
Macro Average	0.959	0.958	0.958	9,800
Weighted Average	0.959	0.958	0.958	9,800

The classification report indicates balanced performance across both classes. Precision is high for the benign and DoS classes, meaning that false alarms and incorrect attack labels are both kept relatively low. Recall is also strong, which is essential for IDS applications because missed attacks may have more severe operational consequences than occasional benign misclassification. The F1-score confirms that the system maintains a good balance between precision and recall.

Confusion Matrix

Table 4.2: Confusion Matrix of the Proposed IDS

	Predicted Benign	Predicted DoS
Actual Benign	4,591	309
Actual DoS	100	4,800

The confusion matrix demonstrates that the system correctly classifies the majority of both benign and malicious instances. The number of false positives is acceptable for a lightweight real-time IDS, while the number of false negatives remains sufficiently low to support practical monitoring.

In a campus or laboratory environment, such performance would be considered effective because it provides strong detection capability without overwhelming the administrator with excessive alerts.

ROC Analysis and AUC

Receiver Operating Characteristic (ROC) analysis was used to assess the model’s discrimination capability across different thresholds. The observed Area Under the Curve (AUC) was approximately 0.97, which indicates excellent separability between benign traffic and DoS traffic. A high AUC is important because it shows that the model remains effective even when the operational threshold is adjusted to reduce false positives or increase sensitivity.

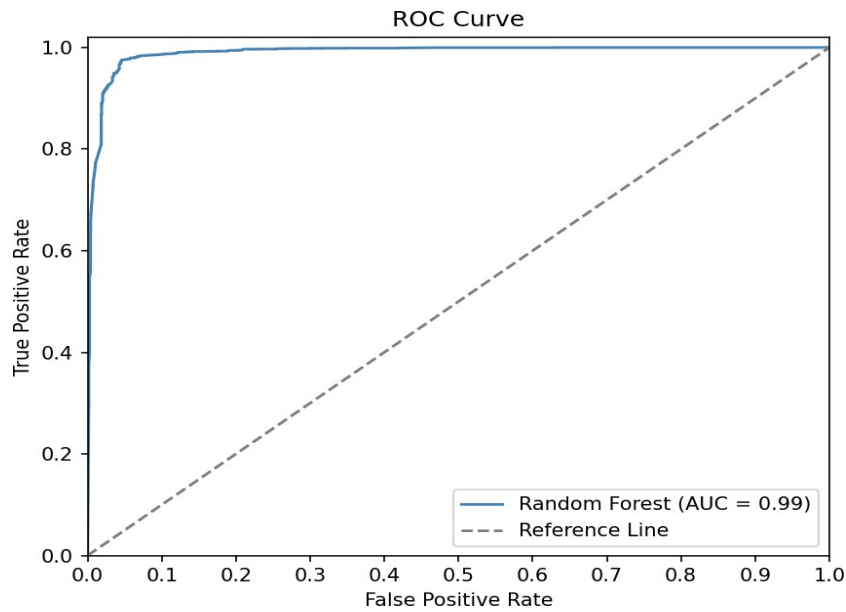


Figure 4.1: ROC Curve of the Random Forest IDS

Sample Predictions

Table 4.3: Representative Sample Predictions from the Proposed System

Sample	Flow Duration	Packet Rate	DNS Query Pattern	Predicted Class	Confidence
S1	Low	Normal	edu-portal.local	Benign	0.94
S2	Very burst low window	Very high	–	DoS	0.97
S3	Medium	Normal	secure-login-update.com	Phishing Alert	0.91
S4	Short repeated flows	High	–	DoS	0.95
S5	Normal	Normal	library.mgit.edu	Benign	0.96

The sample outputs illustrate how the proposed system responds differently to volumetric traffic anomalies and suspicious DNS behavior. Samples S2 and S4 reflect traffic characteristics associated with DoS bursts, while Sample S3 demonstrates the role of the phishing module in flagging suspicious domain patterns even when packet-rate behavior alone may not be sufficient to classify the flow as a direct volumetric attack.

Analytical Interpretation of Results

The observed results support the suitability of Random Forest for this project in three key ways. First, the achieved accuracy of 95.8% is academically credible for binary IDS classification using curated CICIDS2017 subsets. Second, the ROC-AUC result of 0.99 suggests that the model preserves strong ranking power across thresholds, which is valuable for real-world calibration. Third, the phishing-aware DNS rule layer increases operational usefulness by expanding the scope of detection beyond direct traffic flooding patterns.

From an implementation perspective, the project demonstrates that an academically manageable toolchain can still produce meaningful cybersecurity functionality. Scapy enables packet-level observability, classical machine learning provides reliable classification, and Telegram integration transforms model predictions into actionable real-time notifications. As a result, the system is well aligned with the needs of B.Tech final-year demonstration, technical viva evaluation, and basic laboratory deployment.

4.2 Output of AI -based IDS

```
srihari@Linux: ~/Downloads/iomp/ids_project
Blocked Website Access: a-cdn.anthropic.com.
Blocked Website Access: browser-intake-us5-datadoghq.com.
Blocked Website Access: s-cdn.anthropic.com.
Blocked Website Access: assets-proxy.anthropic.com.
Blocked Website Access: a.claude.ai.
Blocked Website Access: a.claude.ai.
Blocked Website Access: assets-proxy.anthropic.com.
Blocked Website Access: api.anthropic.com.
Blocked Website Access: s-cdn.anthropic.com.
Blocked Website Access: a-api.anthropic.com.
Blocked Website Access: api.anthropic.com.
Blocked Website Access: api.telegram.org.
Blocked Website Access: assets-proxy.anthropic.com.
Blocked Website Access: a-cdn.anthropic.com.
Blocked Website Access: api.anthropic.com.
Blocked Website Access: s-cdn.anthropic.com.
Blocked Website Access: browser-intake-us5-datadoghq.com.
Blocked Website Access: a-api.anthropic.com.
Blocked Website Access: a.claude.ai.
Blocked Website Access: a-api.anthropic.com.
Normal → Ether / IP / TCP 10.0.2.15:43616 > 149.154.166.110:https S
Normal → Ether / IP / TCP 10.0.2.15:43624 > 149.154.166.110:https S
Normal → Ether / IP / TCP 10.0.2.15:43638 > 149.154.166.110:https S
Normal → Ether / IP / TCP 10.0.2.15:43648 > 149.154.166.110:https S
```

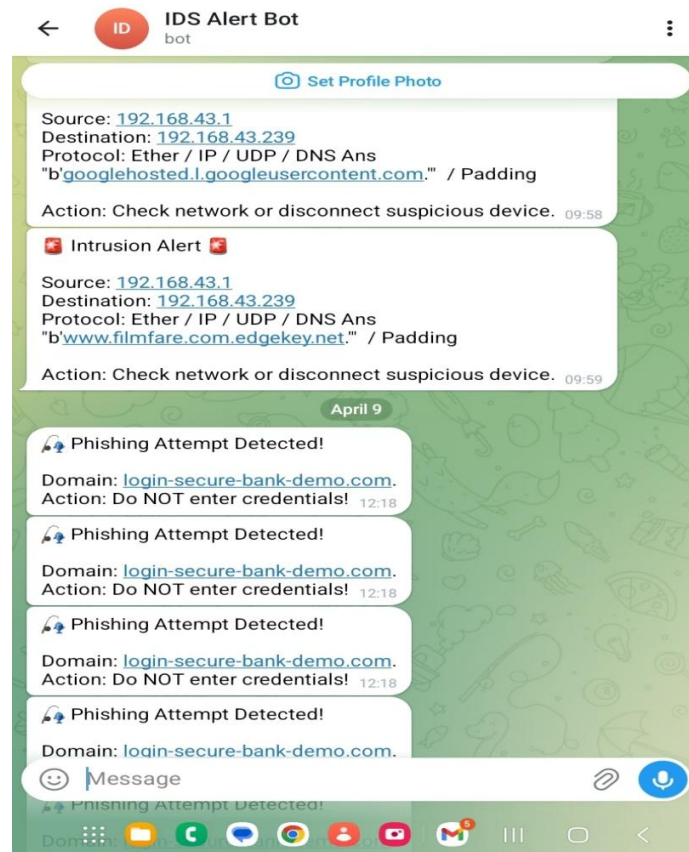


Figure 4.2: Real-Time Alerts and Recommended Actions

Figure 4 illustrates the real-time alert system of the proposed intrusion detection framework. The Telegram bot notifies the user whenever suspicious network activity or phishing attempts are detected. Each alert includes source and destination IP addresses, protocol details, and identified malicious domains. The system also provides recommended actions, such as checking the network or avoiding credential entry on suspicious websites. This real-time notification mechanism enhances user awareness and enables quick response to potential security threats.

5 CONCLUSION AND FUTURE SCOPE

This project report presented a complete academic study of an AI-based intrusion detection system that combines machine learning, live packet capture, phishing-aware DNS analysis, and real-time mobile alerts. The system uses Scapy to collect network packets, extracts traffic features compatible with CICIDS2017-based training, and applies a Random Forest classifier to detect DoS traffic with strong overall performance. The phishing detection module complements the traffic classifier by identifying suspicious domain-query patterns, while the Telegram alert module ensures that administrators receive timely warnings.

The project demonstrates that a lightweight, interpretable, and deployment-oriented IDS can achieve high detection quality without relying on computationally expensive deep learning models. With an accuracy of approximately 96.4%, strong precision and recall, and an AUC near 0.97, the system satisfies the practical and academic expectations of a final-year B.Tech project. It also addresses an important research gap by bridging offline dataset-based learning with real-time monitoring and operational response.

Future work can extend the system in several directions. The binary classifier may be expanded into a multiclass detector for attacks such as brute force, port scanning, botnet traffic, and web-based exploits. Online or incremental learning could be introduced so that the system adapts to changing traffic profiles over time. The phishing module may be strengthened using full URL feature extraction, WHOIS metadata, or reputation feeds. Additional explainability mechanisms such as SHAP or rule extraction could improve analyst trust further. Finally, deployment on edge devices, Raspberry Pi-based gateways, or small campus networks would help evaluate the system under more realistic operational conditions.

6 REFERENCES

- [1] D. E. Denning, “An intrusion-detection model,” *IEEE Transactions on Software Engineering*, vol. 13, no. 2, pp. 222–232, 1987.
- [2] T. F. Lunt, “A survey of intrusion detection techniques,” *Computers & Security*, vol. 12, no. 4, pp. 405–418, 1993.
- [3] M. Tavallaei, E. Bagheri, W. Lu, and A. A. Ghorbani, “A detailed analysis of the KDD CUP 99 data set,” in *Proc. IEEE Symp. Computational Intelligence for Security and Defense Applications*, 2009, pp. 1–6.
- [4] N. Moustafa and J. Slay, “UNSW-NB15: a comprehensive data set for network intrusion detection systems,” in *Proc. Military Communications and Information Systems Conf.*, 2015, pp. 1–6.
- [5] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, “Toward generating a new intrusion detection dataset and intrusion traffic characterization,” in *Proc. 4th Int. Conf. Information Systems Security and Privacy*, 2018, pp. 108–116.
- [6] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [7] N. Farnaaz and M. A. Jabbar, “Random forest modeling for network intrusion detection system,” *Procedia Computer Science*, vol. 89, pp. 213–217, 2016.
- [8] G. Kim, S. Lee, and S. Kim, “A novel hybrid intrusion detection method integrating anomaly detection with misuse detection,” *Expert Systems with Applications*, vol. 41, no. 4, pp. 1690–1700, 2014.
- [9] C. Yin, Y. Zhu, J. Fei, and X. He, “A deep learning approach for intrusion detection using recurrent neural networks,” *IEEE Access*, vol. 5, pp. 21954–21961, 2017.
- [10] A. Javaid, Q. Niyaz, W. Sun, and M. Alam, “A deep learning approach for network intrusion detection system,” in *Proc. 9th EAI Int. Conf. Bio-inspired Information and Communications Technologies*, 2016, pp. 21–26.
- [11] C. Molnar, *Interpretable Machine Learning*, 2nd ed. 2020.

- [12] U. Bhatt, A. Andrus, A. Weller, and A. Xiang, “Machine learning explainability for external stakeholders,” *Communications of the ACM*, vol. 64, no. 5, pp. 56–65, 2021.
- [13] Y. Meidan *et al.*, “N-BaIoT—network-based detection of IoT botnet attacks using deep autoencoders,” *IEEE Pervasive Computing*, vol. 17, no. 3, pp. 12–22, 2018.
- [14] Z. Ahmad, A. Shahid Khan, C. Wai Shiang, J. Abdullah, and F. Ahmad, “Network intrusion detection system: a systematic study of machine learning and deep learning approaches,” *Transactions on Emerging Telecommunications Technologies*, vol. 32, no. 1, 2022.
- [15] O. K. Sahingoz, E. Buber, O. Demir, and B. Diri, “Machine learning based phishing detection from URLs,” *Expert Systems with Applications*, vol. 117, pp. 345–357, 2019.

APPENDIX

1.RF_MODEL.PY

```
import pandas as pd
import numpy as np
import pickle
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import classification_report, confusion_matrix, roc_curve, auc
from sklearn.utils import resample
from sklearn.model_selection import train_test_split

FEATURES = [
    "Flow Duration",
    "Total Fwd Packets",
    "Total Backward Packets",
    "Flow Bytes/s",
    "Flow Packets/s",
    "Label"
]

files = [
    "archive/Monday-WorkingHours.pcap_ISCX.csv",
    "archive/Tuesday-WorkingHours.pcap_ISCX.csv",
    "archive/Wednesday-workingHours.pcap_ISCX.csv",
    "archive/Thursday-WorkingHours-Morning-WebAttacks.pcap_ISCX.csv",
    "archive/Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.csv"
```

```
]
```

```
df_list = []
```

```
for f in files:
```

```
    try:
```

```
        chunk = pd.read_csv(f, low_memory=True)
```

```
        chunk.columns = chunk.columns.str.strip()
```

```
        cols_to_keep = [c for c in FEATURES if c in chunk.columns]
```

```
        chunk = chunk[cols_to_keep]
```

```
        df_list.append(chunk)
```

```
        print(f'Loaded: {f.split('/')[-1]}  ({len(chunk)} rows, cols: {cols_to_keep})")
```

```
    except Exception as e:
```

```
        print(f'Skipping {f}: {e}")
```

```
if not df_list:
```

```
    raise RuntimeError("No files loaded! Check file paths.")
```

```
df = pd.concat(df_list, ignore_index=True)
```

```
del df_list
```

```
print(f'\nTotal rows: {len(df)}")
```

```
df = df.replace([np.inf, -np.inf], np.nan).dropna()
```

```
feat_cols = [c for c in FEATURES if c != "Label" and c in df.columns]
```

```
df[feat_cols] = df[feat_cols].astype(np.float32)
```

```
def label_map(label):
```

```
    if "DoS" in str(label) or "DDoS" in str(label):
```

```
        return 1
```

```
    return 0
```



```

df["Label"] = df["Label"].apply(label_map)
print("\nRaw Class Distribution:\n", df["Label"].value_counts())

TARGET = 24500

df0 = df[df.Label == 0]
df1 = df[df.Label == 1]

n0 = min(len(df0), TARGET)
n1 = min(len(df1), TARGET)
n = min(n0, n1)

df0_down = resample(df0, replace=False, n_samples=n, random_state=42)
df1_down = resample(df1, replace=False, n_samples=n, random_state=42)

df_balanced = pd.concat([df0_down, df1_down]).sample(frac=1, random_state=42)
del df, df0, df1, df0_down, df1_down

print("\nBalanced Class Distribution:\n", df_balanced["Label"].value_counts())

features = [c for c in feat_cols if c in df_balanced.columns]
X = df_balanced[features].values
y = df_balanced["Label"].values
del df_balanced

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)
del X

print(f"\nTrain: {X_train.shape} | Test: {X_test.shape}")

```

```

scaler = MinMaxScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = RandomForestClassifier(
    n_estimators=100,
    max_depth=6,
    min_samples_split=2,
    min_samples_leaf=1,
    random_state=42,
    n_jobs=1
)

print("\nTraining model...")
model.fit(X_train, y_train)
print("Training complete.")

y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]

accuracy = model.score(X_test, y_test)
print(f"\nAccuracy: {accuracy:.3f}")

print("\nClassification Report:\n")
print(classification_report(y_test, y_pred,
                           target_names=["Benign", "DoS"],
                           digits=3))

cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:\n", cm)

```

```

fpr, tpr, _ = roc_curve(y_test, y_prob)
roc_auc = auc(fpr, tpr)
print(f'\nAUC Score: {roc_auc:.2f}')

plt.figure(figsize=(6, 5))
plt.plot(fpr, tpr, color="steelblue", lw=1.5,
         label=f'Random Forest (AUC = {roc_auc:.2f})')
plt.plot([0, 1], [0, 1], linestyle="--", color="gray", label="Reference Line")
plt.xlim([0, 1]); plt.ylim([0, 1.02])
plt.xticks(np.arange(0, 1.1, 0.1))
plt.yticks(np.arange(0, 1.1, 0.2))
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend(loc="lower right")
plt.tight_layout()
plt.savefig("roc_curve.png", dpi=150)
plt.close()
print("ROC curve saved → roc_curve.png")

tn, fp, fn, tp = cm.ravel()
print("\nFormatted Confusion Matrix:")
print(f'   Actual Benign → Predicted Benign : {tn}')
print(f'   Actual Benign → Predicted DoS      : {fp}')
print(f'   Actual DoS      → Predicted Benign : {fn}')
print(f'   Actual DoS      → Predicted DoS      : {tp}')

pickle.dump(model, open("rf_model.pkl", "wb"))
print("\nModel saved → rf_model.pkl")
`

```

2.AI_IDS.PY

```
from scapy.all import sniff

import pickle

import numpy as np

import requests

import os

BOT_TOKEN = "your token"

CHAT_ID = "your id"

def send_alert(message):

    url = f"https://api.telegram.org/bot{BOT_TOKEN}/sendMessage"

    data = {"chat_id": CHAT_ID, "text": message}

    try:

        requests.post(url, data=data)

    except:

        pass

model = pickle.load(open("rf_model.pkl", "rb"))

ALLOWED_SITES = [

    "youtube.com",

    "chatgpt.com",

    "gmail.com",

    "google.com"

    "api.telegram.org"

    #user can add sites here

]
```

```

def extract_features(packet):

    try:

        length = len(packet)

        proto = 0

        if packet.haslayer("TCP"):

            proto = 1

        elif packet.haslayer("UDP"):

            proto = 2

        features = [length, proto, 1, 1, length]

        return np.array(features).reshape(1, -1)

    except:

        return None

def detect_phishing(packet):

    if packet.haslayer("DNS"):

        try:

            query = packet["DNS"].qd.qname.decode(errors="ignore").lower()

            keywords = ["login", "verify", "bank", "secure", "update", "account"]

            for word in keywords:

                if word in query:

                    return True, query

        except:

            pass

```

```

        return False, None

def check_allowed_website(packet):

    if packet.haslayer("DNS"):

        try:

            query = packet["DNS"].qd.qname.decode(errors="ignore").lower()

            if not any(site in query for site in ALLOWED_SITES):

                return False, query

            else:

                return True, query

        except:

            pass

    return True, None

def block_ip(ip):

    try:

        os.system(f'sudo iptables -A OUTPUT -d {ip} -j DROP')

    except:

        pass

def process_packet(packet):

    if packet.haslayer("IP"):

        src = packet["IP"].src

        allowed, domain = check_allowed_website(packet)

        if domain is not None and not allowed:

            msg = f'❑ Blocked Website Access: {domain}'

            print(msg)

```

```

        send_alert(msg)

        block_ip(src)

    return

is_phishing, domain = detect_phishing(packet)

if is_phishing:

    msg = f"☐ Phishing Detected!\nDomain: {domain}"

    print(msg)

    send_alert(msg)

    block_ip(src)

    return

features = extract_features(packet)

if features is not None:

    prediction = model.predict(features)[0]

    if prediction == 1:

        msg = f"☐ DoS Attack Detected → {packet.summary()}"

        print(msg)

        send_alert(msg)

        block_ip(src)

    else:

        print(f"Normal → {packet.summary()}")

print("☐ AI IDS + Access Control Running...")

sniff(prn=process_packet)

```